

L'IA de détection d'anomalies de Sentinel-X

Groupe 1 · EPSI M1 DEV · Workshop Sentinel-X — octobre 2026
Service `ai/anomaly` · Isolation Forest · MQTT · FastAPI

1 Le rôle de l'IA

L'IA surveille en continu les mesures du boîtier (température, humidité, gaz) et détecte tout **comportement inhabituel** par rapport à ce qu'elle a appris comme « normal ».

Elle ne remplace pas l'alarme gaz du boîtier, elle vient **en plus** :

	Alarme du boîtier	IA anomalies
Principe	seuil fixe (gaz $\geq$ 50)	comparaison avec le comportement normal appris
Ce qu'elle voit	une valeur trop haute	une valeur, une tendance ou une combinaison inhabituelle
Où elle tourne	dans l'ESP8266	dans un service Docker sur le PC
Sans réseau	fonctionne	ne fonctionne pas

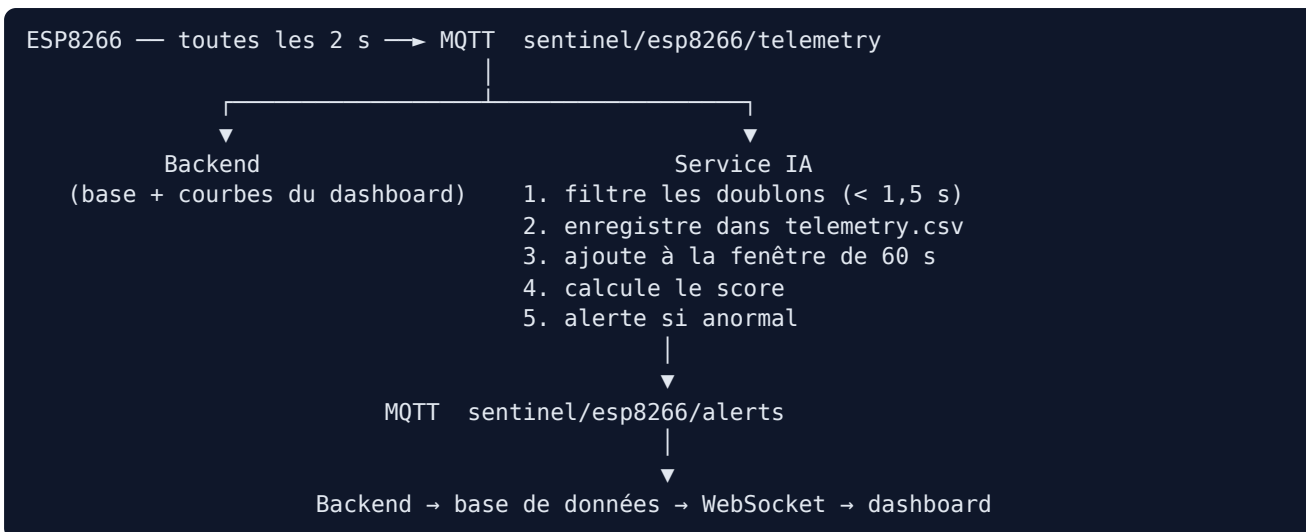
2 Où elle se trouve

C'est un **service indépendant**, dans le dossier `ai/anomaly/` :

Fichier	Rôle
<code>detector.py</code>	le cœur : calcul des indicateurs, entraînement, détection
<code>main.py</code>	réception des mesures (MQTT), enregistrement, API HTTP
<code>data/telemetry.csv</code>	les mesures enregistrées, qui servent à l'entraînement
<code>models/anomaly_model.joblib</code>	le modèle entraîné

Il tourne dans le conteneur `sentinel-anomaly`, sur le port 8002. Le backend n'a **aucun code IA** : il reçoit simplement les alertes comme celles du boîtier.

3 Le trajet d'une mesure



4 La fenêtre glissante

L'IA ne juge jamais une mesure isolée, qui serait trop bruitée. Elle garde **les 30 dernières mesures, soit 60 secondes**, et réévalue cette minute à chaque nouvelle mesure. Une valeur bizarre isolée (bruit du capteur) ne suffit donc pas à déclencher une alerte : il faut que **toute la minute** soit inhabituelle.

5 Les 7 indicateurs

Chaque minute est résumée en 7 nombres, les « caractéristiques » :

Indicateur	Calcul	Ce qu'il détecte
temp	dernière température	une valeur jamais vue
hum	dernière humidité	une valeur jamais vue
gas	dernière valeur de gaz	une valeur jamais vue
temp_slope	pente de la température sur 60 s (régression linéaire)	une température qui monte ou baisse vite
gas_slope	pente du gaz sur 60 s	du gaz qui monte vite
gas_std	écart-type du gaz sur 60 s	un gaz instable , qui varie beaucoup
corr_tg	corrélation température/gaz (de -1 à 1)	température et gaz qui évoluent ensemble

Les 4 derniers sont la vraie force de l'IA : ils détectent un **comportement**, pas seulement une valeur. Par exemple, une température de 27 °C n'a rien d'anormal, mais une température qui gagne 1,5 °C par minute, si.

6 Le modèle : Isolation Forest

Le principe. Le modèle construit 200 arbres qui découpent les données au hasard :

- un point **normal** se trouve au milieu de beaucoup d'autres : il faut **beaucoup** de découpes pour l'isoler ;
- un point **anormal** est seul dans son coin : **peu** de découpes suffisent.

Le nombre moyen de découpes donne le **score** : plus il est bas, plus le point est anormal.

Pourquoi ce modèle :

- Il apprend **sans exemples d'incidents**, uniquement à partir de données normales. C'est indispensable, car on ne peut pas provoquer de vraies fuites de gaz pour lui apprendre.
- Il est léger et rapide : un calcul par mesure, en quelques millisecondes.
- C'est un modèle classique et reconnu pour la détection d'anomalies (bibliothèque scikit-learn).

7 L'entraînement

Il se lance avec le bouton « **Réentraîner le modèle** » du dashboard, ou avec `POST /api/anomaly/train`.

1. Le service prend les **900 dernières mesures** du boîtier (30 min), supposées normales.
2. Il les découpe : **80 %** pour apprendre, **20 %** pour tester.
3. Il calcule les 7 indicateurs pour chaque fenêtre de 60 s.
4. Il entraîne l'Isolation Forest.
5. Il calcule le **seuil** : la valeur sous laquelle tombent seulement 0,2 % des minutes normales.
6. Il **s'auto-évalue** :
 - **fausses alertes** : il rejoue les 20 % de données jamais vues, et il devrait y en avoir 0 ;
 - **dérive simulée** : il ajoute une montée lente de la température et du gaz aux données, puis mesure en combien de secondes l'IA la détecte, comparé au temps qu'il faudrait au seuil fixe « gaz > 600 ».
7. Il enregistre le modèle sur le disque. Le modèle est rechargé automatiquement au redémarrage.

Au premier démarrage, s'il n'existe pas encore de modèle et qu'il y a au moins 150 mesures, l'entraînement se lance tout seul.

8 La détection en temps réel

À chaque nouvelle mesure, toutes les 2 s :

```
score = modèle(7 indicateurs de la dernière minute)

si score < seuil → compteur + 1
sinon          → compteur = 0

compteur atteint 3 (6 s anormales d'affilée) → alerte ANOMALY TRIGGERED
compteur retombe à 0 pendant une alerte     → alerte ANOMALY CLEARED
```

La règle des **3 fois de suite** évite qu'un pic isolé déclenche une alerte. Chaque appareil a son propre compteur, donc on peut ajouter d'autres boîtiers.

9 Le contenu d'une alerte

Elle est publiée sur MQTT `sentinel/esp8266/alerts` :

```
{
  "device_id": "esp8266-01",
  "source": "ml",
  "type": "ANOMALY",
  "state": "TRIGGERED",
  "score": -0.21,
  "detail": {"temp": 27.4, "hum": 44.8, "gas": 44, "temp_slope": 0.05,
             "gas_slope": 0.01, "gas_std": 1.1, "corr_tg": 0.2}
}
```

Le champ `detail` contient les 7 indicateurs : c'est lui qui permet d'**expliquer** l'alerte.

10 L'affichage dans le dashboard

La carte « **IA · Détection d'anomalies** » interroge `/api/anomaly/status` toutes les 3 s et affiche :

- l'état : non entraîné, en préparation, normal ou anomalie ;
- la courbe du score en direct, avec la ligne du seuil et la zone rouge ;
- l'évaluation (fausses alertes, et 12 s contre 520 s pour la dérive) ;
- le bouton d'entraînement.

La liste des alertes affiche chaque anomalie avec son **explication**, calculée à partir de `detail` : « la température monte vite (+1,5 °C/min) », « température et gaz montent ensemble », « le gaz est instable », etc.

11 L'API du service (port 8002)

Route	Rôle
GET <code>/api/anomaly/status</code>	état du modèle, seuil, évaluation, score en direct
POST <code>/api/anomaly/train</code>	entraîne le modèle (paramètre <code>limit</code> = nombre de mesures)
GET <code>/health</code>	le service tourne-t-il, est-il connecté à MQTT ?
<code>/docs</code>	Swagger pour tout tester

12 La gestion des données

- **Enregistrement continu** de chaque mesure dans `telemetry.csv`.
- **Taille limitée** aux 20 000 dernières mesures (environ 11 h), les plus anciennes sont supprimées automatiquement.
- **Doublons filtrés** : le firmware renvoie une mesure en plus à chaque changement du PIR. L'IA ignore les mesures arrivées moins de 1,5 s après la précédente, pour que la fenêtre fasse toujours bien 60 s.

13 Les réglages

Paramètre	Valeur	Effet
Fenêtre	30 mesures (60 s)	durée analysée à chaque fois
CONSECUTIVE	3	nombre de minutes anormales d'affilée avant l'alerte
Quantile du seuil	0,2 %	plus il est bas, moins il y a d'alertes
limit	900	mesures utilisées pour l'entraînement
MAX_ROWS	20 000	taille maximale du fichier de données
MIN_INTERVAL_S	1,5 s	filtre des doublons

14 Les résultats

Avec les données de test, après entraînement :

- **0 fausse alerte** sur des données jamais vues ;
- une dérive lente détectée en **~12-14 s**, contre **~520-700 s** pour un seuil fixe, soit environ **40 fois plus tôt**.

15 Les limites, et comment y répondre

Limite	Réponse
Le modèle considère comme normal tout ce qu'il a vu	l'entraîner au calme, après les 15 min de chauffe du MQ-2
Peu de données : il ne connaît qu'une petite plage de température	laisser tourner plusieurs heures, puis réentraîner avec un <code>limit</code> plus grand
Changement d'environnement (autre pièce, saison)	réentraîner dans le nouvel environnement
Le service tourne sur le PC, pas dans le boîtier	l'alarme gaz du boîtier reste active même sans réseau